

Avaliação do Impacto de Estratégias de Chunking na Qualidade de Sistemas RAG: Um Estudo Comparativo em Domínios Gerais e Agronômicos

Murillo de Moura Ferraz¹, Daniel do Santos Kaster¹

¹Departamento de Computação – Universidade Estadual de Londrina (UEL)
Caixa Postal 10.011 – CEP 86057-970 – Londrina – PR – Brasil

`murillo.ferraz@uel.br`, `dskaster@uel.br`

Abstract. *Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation, but suffer from well-known limitations such as hallucination and static training data. Retrieval-Augmented Generation (RAG) has emerged as a prominent solution to these issues by coupling LLMs with dynamic information retrieval. A fundamental, yet underexplored, component of RAG pipelines is the document chunking strategy, which determines how source texts are segmented before indexing. This work proposes a comprehensive empirical evaluation of three chunking strategies: Fixed-size Chunking, Recursive Character Text Splitting, and Semantic Chunking. Applied to a controlled RAG pipeline. The evaluation is conducted in two phases: (i) validation on a benchmark dataset using the RAGAS framework; and (ii) application to a agronomic domain dataset with agricultural texts and domain-specific questions. This dual-phase approach provides both methodological validation and practical contributions to RAG system design in specialized domains.*

Resumo. *Modelos de Linguagem de Grande Escala (Large Language Models, LLMs) demonstram capacidades notáveis em compreensão e geração de linguagem natural, mas apresentam limitações conhecidas, como alucinação e dependência de dados estáticos de treinamento. A Recuperação Aumentada por Geração (Retrieval-Augmented Generation, RAG) integra esses modelos a mecanismos de recuperação de informação, reduzindo tais limitações. Um componente crítico dos pipelines RAG é a estratégia de chunking, que determina como os documentos-fonte são segmentados antes da indexação. Este trabalho propõe a avaliação empírica de três estratégias de chunking: Fixed-size Chunking, Recursive Character Text Splitting e Semantic Chunking. Aplicadas a um pipeline RAG controlado. A avaliação é conduzida em duas fases: (i) validação no conjunto de dados de um benchmark utilizando o framework RAGAS; e (ii) aplicação a um novo conjunto de dados de domínio agrônomo com textos agrícolas e perguntas específicas do domínio. Essa abordagem de duas fases fornece tanto validação metodológica quanto contribuições práticas para o design de sistemas RAG em domínios especializados.*

1. Introdução

O avanço dos Modelos de Linguagem de Grande Escala (*Large Language Models*, LLMs) transformou diversas áreas da computação, viabilizando aplicações de geração, sumarização e resposta a perguntas em linguagem natural com desempenho antes inatingível [2]. Contudo, esses modelos apresentam limitações estruturais relevantes: tendem a produzir respostas factualmente incorretas, fenômeno denominado *hallucination* [8], operam sobre um conjunto de dados estático definido no momento do treinamento, sem capacidade nativa de incorporar informações atualizadas [10], se perdem em meio a grandes janelas de contexto e fogem do centro de pesquisa.

A Recuperação Aumentada por Geração (*Retrieval-Augmented Generation*, RAG) foi proposta como uma arquitetura que mitiga essas limitações ao combinar um mecanismo de recuperação de informação com a capacidade generativa dos LLMs [10]. Em vez de depender exclusivamente do conhecimento parametrizado no modelo, o RAG recupera trechos relevantes de uma base de documentos externa em tempo de inferência, fornecendo ao LLM contexto atualizado e verificável para a geração da resposta [10, 6, 5].

A qualidade de um sistema RAG depende de múltiplos componentes: o modelo de *embedding* utilizado para representação vetorial dos textos, o banco de dados empregado na indexação e recuperação, o LLM responsável pela geração e a estratégia adotada para segmentar os documentos de origem em fragmentos menores, processo denominado *chunking* [5]. A granularidade e o método de partição impactam diretamente a qualidade do contexto recuperado, influenciando a fidelidade e a relevância das respostas geradas pelo LLM [18].

Apesar de sua importância central no desempenho dos sistemas RAG, a estratégia de *chunking* permanece como um dos aspectos menos sistematicamente estudados na literatura. A maioria dos trabalhos recentes trata o *chunking* como etapa secundária ou simplesmente adota parâmetros fixos sem justificativa empírica [1]. Este trabalho propõe uma avaliação empírica comparativa de três estratégias de *chunking*: *Fixed-size Chunking*, *Recursive Character Text Splitting* e *Semantic/Embedding-based Chunking*. A avaliação utilizará o *framework* RAGAS [4], que oferece métricas automatizadas amplamente adotadas para sistemas RAG.

Entretanto, observa-se que a maioria dos estudos que avaliam estratégias de *chunking* o fazem em conjuntos de dados genéricos contêm artigos de enciclopédia de amplo domínio. Domínios especializados, como a agricultura e o agronegócio, apresentam características linguísticas, estruturais e semânticas distintas dos textos gerais. Esses domínios contêm textos técnicos, contrastando bases comuns, além de relações entre conceitos que diferem significativamente. Investigar como estratégias de *chunking* se comportam quando aplicadas a textos agronômicos é essencial para o desenvolvimento de sistemas RAG eficazes em domínios de aplicação prática crítica aplicada ao cenário agrícola.

Assim, este trabalho adota uma metodologia composta por duas fases. Na primeira fase, valida-se a robustez das estratégias em um benchmark da área. Na segunda fase, o conhecimento obtido é transferido para um novo benchmark em domínio agronômico. Essa abordagem permite contribuir tanto para a compreensão fundamental das estratégias quanto para aplicações práticas em contextos especializados.

O restante deste documento está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórico-metodológica e o estado da arte; a Seção 3 descreve os objetivos do trabalho; a Seção 4 detalha os procedimentos metodológicos; a Seção 5 apresenta o cronograma de execução; e a Seção 6 discute as contribuições e resultados esperados.

2. Fundamentação Teórico-Metodológica e Estado da Arte

Esta seção apresenta os conceitos, técnicas e métodos necessários ao desenvolvimento da pesquisa proposta. A organização segue uma progressão do geral ao específico: dos fundamentos dos LLMs ao *pipeline* RAG, passando por *embeddings*, bancos de dados vetoriais, estratégias de *chunking* e métricas de avaliação.

2.1. Modelos de Linguagem de Grande Escala

Os Modelos de Linguagem de Grande Escala (LLMs) são redes neurais profundas fundamentadas na arquitetura *Transformer* [17], que utiliza mecanismos de auto-atenção (*self-attention*) para capturar dependências de longo alcance em sequências de texto de forma paralelizável. Esses modelos são treinados por meio de aprendizado auto-supervisionado sobre volumes massivos de texto, com o objetivo de modelar a distribuição estatística da linguagem natural [14].

Após o pré-treinamento, os LLMs acumulam conhecimento de forma paramétrica, codificado nos pesos da rede. Com o aumento da escala de parâmetros e dados de treinamento, observam-se capacidades emergentes, como raciocínio encadeado e aprendizado em contexto (*in-context learning*), que permitem ao modelo realizar tarefas a partir de poucos exemplos sem atualização de pesos [2, 19].

Essa natureza paramétrica, no entanto, impõe limitações práticas. O conhecimento é estático, limitado ao corte temporal dos dados de treinamento (*knowledge cutoff*), e propenso a alucinações [8]. O problema de alucinação decorre, em parte, do fato de o modelo produzir saídas com base em plausibilidade estatística, sem mecanismo nativo de verificação factual. Estratégias como o ajuste fino supervisionado (*supervised fine-tuning*) e o aprendizado por reforço com feedback humano (*RLHF*) foram propostas para reduzir esse comportamento [13], mas não o eliminam.

Ademais, LLMs tendem a subutilizar informações posicionadas no meio de contextos longos, fenômeno denominado *Lost in the Middle* [11]. Esse comportamento implica que a posição e a coesão dos fragmentos fornecidos como contexto influenciam diretamente a qualidade da resposta gerada, o que torna a estratégia de *chunking* especialmente relevante [5].

2.2. Recuperação Aumentada por Geração (RAG)

O paradigma RAG foi introduzido como uma arquitetura que integra recuperação de informação à geração textual [10]. O *pipeline* RAG típico opera em três etapas: (i) **indexação**, em que documentos são segmentados em *chunks*, codificados como vetores por um modelo de *embedding* e armazenados em um banco de dados vetorial; (ii) **recuperação**, em que a consulta do usuário é codificada e os *chunks* semanticamente mais próximos são recuperados via busca por similaridade; e (iii) **geração**, em que os *chunks* recuperados são fornecidos como contexto ao LLM, que produz a resposta final.

Essa arquitetura reduz a dependência do conhecimento parametrizado, possibilita o acesso a informações atualizadas e permite maior rastreabilidade das fontes utilizadas na geração [7]. Por essas razões, o RAG tornou-se uma das abordagens mais adotadas em aplicações práticas de LLMs.

O *Modular RAG* decompõe o *pipeline* em componentes intercambiáveis, facilitando a experimentação isolada de cada etapa[6]. Essa perspectiva é diretamente alinhada com a metodologia adotada neste trabalho, que mantém todos os componentes fixos e varia exclusivamente a estratégia de *chunking*.

Os pontos de falha mais comuns em sistemas RAG foram catalogados e identificados em sete categorias, incluindo fragmentação inadequada d contexto, recuperação de trechos irrelevantes e alucinações originadas de contextos ruidosos[1]. Esses resultados reforçam a importância de se estudar isoladamente cada componente do *pipeline*, em especial o *chunking*.

2.3. Representação Vetorial de Textos

A representação vetorial de textos, ou *embeddings*, é o mecanismo que viabiliza a recuperação semântica em sistemas RAG. Um modelo de *embedding* mapeia um trecho de texto para um vetor de alta dimensão em um espaço latente, de modo que textos semanticamente similares sejam representados por vetores próximos segundo alguma métrica de distância, sendo a similaridade de cosseno a mais utilizada [15].

Os modelos baseados na arquitetura *Sentence-BERT* [15] foram responsáveis por avanços significativos na qualidade dos *embeddings* para tarefas de recuperação, ao utilizar redes siamesas com *pooling* sobre saídas do BERT para produzir representações densas de sentenças. Modelos mais recentes, como os da família BGE (*BAAI General Embedding*) [3], estendem essa abordagem com técnicas de treinamento contrastivo que melhoram a separação entre pares relevantes e irrelevantes.

A qualidade dos *embeddings* é especialmente sensível ao comprimento e à coesão dos fragmentos codificados. Fragmentos excessivamente curtos podem perder contexto necessário para uma representação precisa; fragmentos muito longos podem diluir a informação relevante. Essa relação direta entre granularidade dos *chunks* e qualidade da representação vetorial é um dos fundamentos que justificam o estudo das estratégias de *chunking* [16].

2.4. Bancos de Dados Vetoriais

Os bancos de dados vetoriais são estruturas de armazenamento e indexação projetadas para suportar buscas eficientes por similaridade em espaços de alta dimensão. Em um *pipeline* RAG, os vetores gerados pelo modelo de *embedding* são armazenados nessas estruturas, e a recuperação é realizada por meio de algoritmos de busca aproximada ao vizinho mais próximo (*Approximate Nearest Neighbor*, ANN), como HNSW (*Hierarchical Navigable Small World*) [12] e IVF (*Inverted File Index*).

Soluções como FAISS [9], Chroma e Weaviate são amplamente adotadas em ambientes acadêmicos e de produção, oferecendo diferentes compromissos entre velocidade de consulta, precisão de recuperação e escalabilidade. Neste trabalho, o banco de dados vetorial é mantido como componente fixo do *pipeline*, de modo que sua influência não interfira na comparação entre estratégias de *chunking*.

2.5. Estratégias de Chunking

O *chunking* é o processo de divisão dos documentos-fonte em fragmentos menores antes da indexação. A escolha da estratégia influencia diretamente a granularidade semântica dos fragmentos recuperados, o que afeta tanto a relevância do contexto fornecido ao LLM quanto a fidelidade das respostas geradas [5]. As três estratégias avaliadas neste trabalho são descritas a seguir.

2.5.1. Fixed-size Chunking

A segmentação de tamanho fixo divide o texto em fragmentos com um número predefinido de *tokens* ou caracteres, com ou sem sobreposição (*overlap*) entre fragmentos consecutivos. É a abordagem mais simples e computacionalmente eficiente, utilizada como *baseline* em benchmarks [18]. Sua principal limitação está na insensibilidade à estrutura semântica do texto: a divisão arbitrária pode fragmentar sentenças ou parágrafos coesos, reduzindo a relevância contextual dos *chunks* recuperados [1]. O parâmetro de *overlap* parcialmente mitiga esse problema ao garantir sobreposição entre fragmentos consecutivos, mas não resolve a fragmentação semântica em si.

2.5.2. Recursive Character Text Splitting

O *Recursive Character Text Splitter* realiza a segmentação de forma hierárquica, aplicando uma lista ordenada de separadores naturais do texto: tenta primeiro dividir por parágrafos ($\backslash n \backslash n$), depois por quebras de linha ($\backslash n$), depois por pontuação de fim de sentença, e assim por diante, até que os fragmentos atinjam o tamanho-alvo. Essa abordagem preserva a coesão textual em maior grau do que o *fixed-size chunking*, por respeitar fronteiras linguísticas naturais [18]. É a técnica mais empregada em ambientes de produção com a biblioteca LangChain [5], o que a torna um ponto de comparação de relevância prática direta.

2.5.3. Semantic/Embedding-based Chunking

O *chunking* semântico utiliza modelos de *embedding* para decidir os pontos de corte no texto. Uma abordagem representativa, calcula a similaridade de cosseno entre sentenças consecutivas e identifica quedas abruptas de similaridade como fronteiras entre *chunks*. Isso garante que cada fragmento resultante seja internamente coeso do ponto de vista semântico. Embora mais custoso computacionalmente, esse método tende a produzir fragmentos mais informativos para a etapa de recuperação, especialmente em textos com estrutura temática variada.

2.6. Avaliação de Sistemas RAG

A avaliação de sistemas RAG apresenta desafios específicos, pois é necessário mensurar tanto a qualidade da recuperação quanto a qualidade da geração. Métricas tradicionais de recuperação de informação, como precisão e revocação, não capturam adequadamente a adequação do contexto recuperado para a geração [16]. Por outro lado, métricas de

geração de linguagem natural, como BLEU e ROUGE, avaliam similaridade superficial com referências e não verificam fidelidade factual.

O *framework* RAGAS [4] foi desenvolvido especificamente para preencher essa lacuna, oferecendo um conjunto de métricas automatizadas que utilizam o próprio LLM como avaliador (*LLM-as-a-judge*):

- **Faithfulness**: mede se as afirmações presentes na resposta gerada estão ancoradas no contexto recuperado, penalizando alucinações. O cálculo envolve a decomposição da resposta em afirmações atômicas e a verificação de cada uma contra o contexto.
- **Answer Relevance**: avalia se a resposta é pertinente à pergunta formulada, penalizando respostas incompletas ou tangenciais.
- **Context Relevance**: mede a proporção do contexto recuperado que é de fato relevante para responder à pergunta, penalizando a recuperação de fragmentos com ruído.
- **Context Recall**: avalia se o sistema recuperou todos os fragmentos necessários para construir a resposta correta, requerendo *ground truth* anotado.

2.7. RAG em Domínios Especializados: O Caso da Agronomia

Embora sistemas RAG tenham se consolidado como uma abordagem promissora para tarefas gerais de resposta a perguntas, sua aplicação em domínios especializados ainda é uma área pouco explorada. Domínios como medicina, direito e agronomia apresentam características e desafios que diferem significativamente de aplicações genéricas. Entre esses desafios encontram-se a terminologia técnica densa e altamente contextual, a necessidade crítica de precisão factual e o volume limitado de dados anotados disponíveis para pesquisa.

No contexto agrícola brasileiro, sistemas RAG apresentam grande potencial para viabilizar consultas eficientes sobre recomendações técnicas. Essas informações são críticas para pequenos e médios produtores. No entanto, o design e a otimização de componentes específicos de RAG, como a estratégia de chunking, ainda não foram investigados de forma sistemática para textos agrícolas.

3. Objetivos

O objetivo geral deste trabalho é avaliar o impacto de diferentes estratégias de segmentação de documentos (chunking) na qualidade das respostas geradas por sistemas RAG, analisando a consistência do desempenho dessas técnicas tanto em bases de conhecimento gerais quanto em domínios técnicos especializados da agronomia

4. Procedimentos Metodológicos

4.1. Visão Geral

A metodologia adota um delineamento experimental controlado: mantém-se fixo todo o *pipeline* RAG e varia-se exclusivamente a estratégia de *chunking*. Essa abordagem isola a variável de interesse e permite atribuir diferenças de desempenho diretamente à estratégia de segmentação adotada, sem interferência de outros componentes.

4.2. Duas Fases Experimentais

O experimento é organizado em duas fases complementares. A primeira fase utiliza o conjunto de dados de um benchmark e constitui a validação metodológica do experimento. Essa fase permite comparação com resultados disponíveis na literatura e a validação da robustez das três estratégias de chunking em um benchmark amplamente consolidado. Os resultados obtidos nesta fase estabelecem a linha de base para a comparação com a segunda fase.

A segunda fase, por sua vez, representa a contribuição principal deste trabalho. Nela, um novo dataset é criado contendo textos agronômicos acompanhados de perguntas específicas do domínio e suas respostas devidamente anotadas. O mesmo pipeline RAG e as mesmas métricas RAGAS são aplicados, permitindo investigar como as estratégias de chunking se transferem para um domínio especializado.

A comparação entre as duas fases fornece insights relevantes sobre a generalização das estratégias de chunking e seu comportamento em contextos linguísticos e estruturais distintos.

4.3. Pipeline RAG Fixo

O *pipeline* será implementado em Python, utilizando bibliotecas de orquestração (como LangChain ou frameworks similares) que serão selecionadas conforme a compatibilidade as estratégias adotadas durante a fase de implementação. Os componentes fixos serão: o modelo de *embedding*, o banco de dados vetorial, o LLM gerador e o número de *chunks* recuperados por consulta (*top-k*). As escolhas específicas de modelos e ferramentas serão definidas na fase de implementação, priorizando alternativas de código aberto para reprodutibilidade.

4.4. Datasets

Para a primeira fase, utiliza-se um benchmark da área. Esse conjunto de dados contém perguntas, contextos e respostas de referência (ground truth), permitindo o cálculo de todas as métricas RAGAS. Os critérios para a seleção do subconjunto do benchmark serão estabelecidos após uma análise exploratória do dataset. Para a segunda fase, um novo dataset será criado com textos de domínio agronômico. Para medir o impacto das técnicas aplicadas a um contexto mais técnico e especializado.

4.5. Estratégias de Chunking e Parâmetros

As três estratégias serão configuradas da seguinte forma:

1. **Fixed-size Chunking:** O tamanho da janela do *chunk size* e o *overlap* serão definidos a partir de testes preliminares;
2. **Recursive Character Text Splitting:** *chunk size* alvo de 512 *tokens*, com separadores hierárquicos (`\n\n`, `\n`, `.`), conforme implementação padrão do LangChain;
3. **Semantic Chunking:** baseado em similaridade de cosseno entre sentenças consecutivas, utilizando critérios de similaridade semântica para a definição dos pontos de corte, onde os parâmetros e limiares serão refinados durante a fase inicial de experimentação.

4.6. Protocolo de Avaliação

A avaliação será realizada em etapas sequenciais. Na primeira fase, realiza-se a segmentação dos textos, sua indexação, e coleta das métricas RAGAS para cada uma das estratégias de chunking investigadas.

Na segunda fase, aplica-se o mesmo protocolo ao dataset agronômico, mantendo-se as mesmas configurações do pipeline RAG utilizado na primeira fase.

Os resultados são analisados e comparados em duas dimensões. Primeiro, dentro de cada fase, comparam-se os resultados entre as estratégias de chunking utilizando-se análise estatística apropriada. Depois, entre as fases, comparam-se os resultados de cada estratégia obtidos no benchmark com aqueles obtidos no dataset agronômico. Essa comparação entre domínio permite identificar diferenças de desempenho.

Esse formato de avaliação permite que o processo seja revisado e ajustado de acordo com dificuldades técnicas ou inconsistências nos dados que possam surgir durante os testes.

5. Cronograma de Execução

Atividades:

- 1. Revisão bibliográfica aprofundada (LLMs, RAG, *chunking*, RAGAS);
- 2. Definição e configuração do ambiente de desenvolvimento e do *pipeline* RAG fixo;
- 3. Implementação das três estratégias de *chunking*;
- 4. Preparação e pré-processamento do *dataset* do benchmark;
- 5. Execução dos experimentos Fase 1 (Benchmark) e coleta dos resultados;
- 6. Construção do dataset agronômico (coleta e anotação de textos);
- 7. Execução dos experimentos Fase 2 (domínio agronômico) e coleta dos resultados;
- 8. Análise estatística e comparação dos resultados entre as duas fases;
- 9. Redação da monografia;

Tabela 1. Cronograma de Execução

	abr	mai	jun	jul	ago	set	out	nov
Atividade 1	•	•	•					
Atividade 2	•	•						
Atividade 3		•	•					
Atividade 4		•	•					
Atividade 5			•	•				
Atividade 6			•	•	•			
Atividade 7				•	•	•		
Atividade 8					•	•		
Atividade 9			•	•	•	•	•	•

6. Contribuições e/ou Resultados Esperados

Propõe-se uma análise empírica e controlada do impacto das estratégias de chunking sobre a qualidade de sistemas RAG em contextos tanto genéricos quanto especializados. O trabalho consolida um protocolo de avaliação reproduzível para chunking em RAG, utilizando o RAGAS como framework de referência. Diferentemente de trabalhos anteriores que avaliam chunking principalmente em datasets genéricos, este trabalho investiga explicitamente a transferência de estratégias para domínios especializados. Ademais, os resultados deste trabalho fornecerão orientações específicas, com base em um benchmark, sobre qual estratégia de chunking é mais adequada em diferentes cenários, desde sistemas genéricos até domínios altamente especializados.

7. Espaço para Assinaturas

Londrina, *16 de Abril de 2026.*

Aluno

Orientador

Referências

- [1] Scott Barnett et al. Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM Conference on Software Engineering*, 2024.
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [3] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. 2024.
- [4] Shahul Es et al. Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of EACL: System Demonstrations*, 2024.
- [5] Paulo Finardi, Leonardo Avila, Rodrigo Castaldoni, et al. The chronicles of rag: The retriever, the chunk and the generator. *arXiv preprint arXiv:2401.07883*, 2024.
- [6] Yunfan Gao et al. Modular rag: Transforming rag systems into customizable modules. *Proceedings of AAI*, 2024.
- [7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024.
- [8] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [9] Jeff Johnson, Matthijs Douze, and Herve Jegou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [11] Nelson F. Liu et al. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.
- [12] Yury A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [13] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- [14] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI Blog*, 2018.

- [15] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.
- [16] Alireza Salemi and Hamed Zamani. Evaluating retrieval quality in retrieval-augmented generation. In *Proceedings of SIGIR*, 2024.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- [18] Xiaohui Wang et al. Searching for best practices in retrieval-augmented generation. In *Proceedings of EMNLP*, 2024.
- [19] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.